

COL 7364/764 Assignment 1 - Sparse Retrieval

2026 - Version 1.0
(dated 19th August 2026)

Deadline

Submission of the complete implementation, and the report on the algorithms is due on **September 3, 2023, 11:59 PM**. All submissions are to be made on Moodle.

Instructions

Follow all instructions. Submissions not following these instructions will not be evaluated.

1. This programming assignment is to be done by each student individually. Do not collaborate by sharing code, algorithm, and any other pertinent details with each other. You are free to discuss and post questions to seek clarifications. Please post all discussions related to this assignment with "hw1" tag on Piazza.
2. All programs have to be written either using Python or C/C++ programming languages only. Anything else requires explicit prior permission from the instructor. The machine we use to evaluate your submissions has the following versions:
 - **C, C++:** gcc12.3;
 - **Python:** version 3.12.

We recommend you to use same versions to avoid the use of deprecated/unsupported functions or take care to ensure required compatibility.

3. A single zip of the source code has to be submitted. The zip file should be structured such that, upon deflating all submission files should be under a directory with the student's registration number. E.g., if a student's registration number is 20XXYYYY999 then the zip submission should be named 20XXYYYY999.zip. Upon deflating, **all contained files** should be under a directory named ./20XXYYYY999 only (names should be in uppercase). Your submission might be rejected and not be evaluated if you do not adhere to these specifications.

Apart from source files, the submission zip file should contain a build mechanism *if needed* . It is the responsibility of each student to ensure that it compiles and generates the necessary

executable as specified. **Please include an empty file in case building is not required** (*e.g. if you are using Python*).

4. Do not submit notebooks – all programs will be run from the commandline only.
The list of files to be submitted, along with their naming conventions and call signatures as given in the instructions below.
5. **Note that we will use only Ubuntu Linux machines to build and run your assignments.** So take care that your file names, paths, argument handling etc. are compatible.
6. You *should not* submit data files. If you are planning to use any other “special” library, please talk to the instructor first (or post on Teams).
7. Note that your submission will be evaluated using larger collection. Only assumption you are allowed to make is that all documents will be in English, have sentences terminated by a period, and all documents are in one single directory. The overall format of documents will be same as given in your training. The collection directory will be given as an input (see below).
8. **Note that there will be no deadline extensions. Apart from the usual “please start early” advise, I must warn you that this assignment requires significant amount of implementation effort, as well as some ‘manual’ tuning of parameters to get good speed up and performance. Do not wait till the end.**

The file structure in each submission must be exactly as follows (DO NOT RENAME ANY DIRECTORY / FILE that is already shared):

```
20XXYYYY999/
├── conftest.py
├── data/
│   ├── README.md
│   └── toy/
├── Dockerfile
├── docs/
│   ├── DOCKER_SUBMISSION.md
│   └── SUBMISSION_INTERFACE.md
├── harness/
├── README.md
├── requirements.txt
├── scripts/
├── submission/
└── tests/
```

1 Assignment Description

The goal of this assignment is to build and tune a from-scratch inverted-index retriever to top the class leaderboard. You will build a from-scratch inverted-index retrieval engine and tune it to outrank every other submission in the class — and the instructor’s own reference system — on $n\text{DCG}@10$ over a held-out query set. This is the assignment where the difference between a B-grade IR system and an A-grade one is entirely in the details: stemming choices, BM25 parameter tuning, term-weighting design, and how you handle short queries and rare terms.

1.1 Learning Objectives

- Implement an inverted index from raw documents through tokenisation, stopword handling, and postings construction.
- Implement and compare at least two ranking functions: a Boolean/vector-space model and BM25 (Robertson & Walker et al., 1992; Robertson & Zaragoza, 2009).
- Understand, in a hands-on way, why parameter choices (BM25’s k_1 , b) materially change ranking quality on real query sets.
- Correctly implement and interpret standard TREC-style evaluation metrics ($\text{MAP}@k$, $n\text{DCG}@k$, MRR, $\text{Precision}@k$) using `trec_eval`-compatible qrels.
- Experience, first-hand, how leaderboard-driven tuning can overfit to a public dev set — and why a held-out private evaluation set exists.

1.2 What you must implement

You must build the following components yourself — implemented from scratch, without using an existing search library (Lucene/Elasticsearch/Pyserini/Whoosh, etc.) for indexing or scoring — and exposed through the required Python entrypoint (Section 1.3). You do not have to write pure Python: a C/C++ extension you compile yourself (via Cython or pybind11) and call from that entrypoint is fine, provided the indexing/scoring logic is your own; see the starter repo’s `docs/SUBMISSION_INTERFACE.md` (“Compiled extensions”) for the build details and a timing caveat. You may use standard libraries for tokenisation/stemming (e.g., NLTK’s Porter stemmer) and numerical libraries (NumPy).

Indexer: builds an inverted index (postings lists with term frequencies and document/collection statistics needed for your scoring functions) over the provided corpus, and persists it to disk in a format your own code can reconstruct later without rebuilding from the raw corpus — the exact save/load contract is given in the starter repo. The on-disk size of what you persist is itself a graded leaderboard component (Section 7); a naive dump of every field you touched during indexing will cost you there, so decide deliberately what actually needs to survive to query time.

Boolean/VSM retriever: supports conjunctive/disjunctive Boolean queries and a cosine-similarity vector-space ranking with a TF-IDF weighting scheme of your choice.

Probabilistic retriever: a BM25 implementation with tunable k_1 and b .

A combined/custom scorer (optional, for competitive edge): any linear or non-linear combination of the above signals, additional features such as proximity/bigram overlap, or your own heuristic — this is where separation in the leaderboard tends to happen.

1.3 What counts as your "entry"

Your competition entry is three functions —

```
'build_index(corpus_path, index_dir)',
```

```
'load_index(index_dir)', and
```

```
'retrieve(query: str, k: int) -> List[(doc_id, score)]'
```

(exact signatures given in the starter repo) — that the grading harness calls for every held-out query.

Please note that `'build_index'` and `'load_index'`/`'retrieve'` are run by the harness as two separate processes, in that order, with nothing shared between them but what you wrote to `'index_dir'`: this is how the harness verifies your persistence is real rather than an artifact of both calls happening to run in the same Python process. You may internally select whichever of your scorers performs best, or blend them — the harness only sees the final ranked list.

2 Submission Interface & Conformance Checking

Every submission must run through the exact `'build_index(corpus_path, index_dir)'` / `'load_index(index_dir)'` / `'retrieve(query: str, k: int) -> List[(doc_id, score)]'` signatures given in the starter repository — no other calling convention, CLI wrapper, or output format is accepted. The starter repo already contains a trivial reference submission (returns the first k documents in collection order, with working persistence to and from disk) that passes the grading harness end-to-end; fork it and replace the internals rather than building submission plumbing from scratch.

Continuous conformance checking. At the end of each day after 4days from the assignment release date (at 5PM exactly), all submissions are collected (including the old ones), we will run the public harness on a small sanity query set and report pass/fail plus latency on the leaderboard website. Use this to catch interface breakage, not at the deadline.

Containerisation. Submissions run inside the provided Docker image with pinned dependencies; do not assume any package beyond what the image includes without updating the provided `'requirements.txt'` and re-verifying the CI check passes.

Compiled extensions. You are not required to write pure Python — a C/C++ extension you compile yourself (e.g. via Cython or pybind11) and call from `'submission/retrieve.py'` is permitted, provided the underlying indexing/scoring logic is your own (the "no existing library" rule is about which libraries you use, not which language you use). The provided Docker image includes a C/C++ toolchain (`'build-essential'`) so such an extension builds

correctly during ‘docker build’; build it there, not inside ‘`build_index()`’, since anything ‘`build_index()`’ does is charged against your index-build-time efficiency metric.

Conformance freeze. 48 hours before the first submission deadline, the CI check must be green. Interface failures reported for the first time after the freeze are graded as a submission error, not treated as a harness bug.

3 Dataset and Tools

Corpus: a mid-sized TREC-style newswire or web collection (e.g., a subset of Robust04 or TREC-COVID via ‘`ir_datasets`’) — small enough to index on a laptop in minutes, large enough ($\geq 500K$ documents) that naive approaches are noticeably slow.

Training/dev topics + qrels: released with the assignment for local tuning.

Held-out topics: withheld until the private leaderboard is scored at the deadline.

Reference tooling (for comparison only, not for your submission): ‘`ir_datasets`’ for corpus/qrels loading and ‘`trec_eval`’ (or ‘`pytrec_eval`’) for metric computation. The starter repo also ships a script that generates your own local BM25 comparison run using a standard library, with generic textbook parameters — useful for sanity-checking your system, but note it is not the same run, and not necessarily the same parameters, course staff score you against.

4 Competition Mechanics and Scoring

Format: open leaderboard, unlimited submissions before the deadline, public dev-topic score visible in real time, private held-out-topic score revealed at the deadline (standard two-tier design to discourage dev-set overfitting).

Rank component	Metric	Weight
Primary	nDCG@10 on held-out topics	70% of leaderboard score
Secondary (tie-break)	MAP@10 on held-out topics	10%
Efficiency bonus/penalty	Index build time + mean query latency, relative to class median	$\pm 10\%$
Index size	On-disk size of your persisted index after build, relative to class median (full marks at $1/2$ the median, zero at $\geq 2\times$ the median, linear between)	10%

A note on compiled extensions: if you use a C/C++ extension, expect a real advantage on the efficiency and index-size components above over an equivalent pure-Python submission — that trade is expected, not a scoring bug; it mirrors how real IR engines are built and is part of what this section is measuring.

A note on "MAP@10": since `retrieve()` only ever returns up to k documents, this is not textbook unbounded MAP — full MAP would require ranking (or at least scoring) the entire corpus so precision at every rank where a relevant document appears is known. MAP@10 is Average Precision computed over your top 10 results, still normalised by the true total number of relevant documents per query (from the `qrels`, not from what you found) — so a query with more than 10 relevant documents can never reach 1.0 on this metric, for any submission. This matches how `trec_eval` itself behaves on a depth-limited run file; every submission is held to the same yardstick, so relative ranking is unaffected.

Baseline you must beat: a competitive BM25 reference, tuned by course staff on data you do not have access to. Its exact parameters are not disclosed, and are deliberately not the textbook defaults used by the comparison script in the starter repo — matching "the standard values" is not a reliable way to clear this floor. Beating the baseline is the pass/fail floor, not the goal: it exists so that a submission that technically runs but ranks no better than a generic BM25 doesn't pass on leaderboard performance. The competitive-grade component is driven by your percentile rank among the whole class on the private held-out set — the difference between clearing the floor and placing in the top decile is entirely about how much further you push, and the floor tells you nothing about where that ceiling is.

5 Oral Defense

After the private leaderboard closes, a random selection of teams sit a 10–15 minute oral defense: a walkthrough of one design decision, 2–3 targeted questions on your own code, and a live perturbation (the examiner changes a parameter such as k_1 or the smoothing constant and asks you to predict the effect before re-running). If the team cannot explain or meaningfully modify their own submission, the leaderboard-performance component below is capped at 50% of its earned value regardless of rank.

6 What is to be submitted?

1. Source code implementing the indexer and all required retrievers, with a README describing how to reproduce your index and run the harness.
2. A short report (3–4 pages) covering: your indexing design choices (stopwords, stemming, and how you compressed or pruned what you persist to disk etc.), a table comparing Boolean/VSM vs. BM25 performance on the dev set, your parameter search procedure for k_1 and b (include a plot of $nDCG@10$ vs. the swept parameter), and an error analysis of

3–5 queries where your system ranks poorly. This report can be submitted upto 3 days after the competition closes.

3. A one-paragraph description of your final competition entry (which scorer or blend you submitted, and why).
4. An AI-use disclosure statement and a code provenance statement.

7 Grading policy

Component	Weight	Criteria
Leaderboard performance	50%	The Section 7 leaderboard score (nDCG@10 + MAP@10 + efficiency modifier + index-size score), scaled primarily by percentile rank among the whole class on the private held-out set; clearing the undisclosed baseline is a pass/fail floor, not a target to stop at. Capped at 50% of earned value if the oral defense (below) indicates the team cannot explain their own submission.
Correctness of required components	20%	Boolean/VSM and BM25 retrievers are both correctly implemented and independently verifiable (graded by unit tests against known small examples, not leaderboard score alone).
Report quality	20%	Parameter-tuning methodology is sound (e.g., tuned on dev, not held-out); analysis is specific and evidenced, not generic.
Interface conformance	5%	CI smoke test passing at the conformance freeze; submission runs unmodified through the harness.
Code quality & reproducibility	5%	Runs end-to-end from the README; index build and load are both deterministic.

8 Academic Integrity

No existing search/indexing library may be used for the required components. Pretrained language models or embeddings are out of scope for this assignment — that machinery is introduced later in the course. Discussing high-level strategy with classmates is fine; sharing code or a tuned parameter file is not.

LLM assistance is permitted but must be declared in an AI-use disclosure statement, and any external code consulted (including Pyserini source, public repos, or prior submissions) must be listed in a code provenance statement with the specific delta you implemented. Course staff run similarity detection across current- and prior-term submissions and public repositories; flagged submissions receive closer scrutiny at the oral defense rather than an automatic penalty. A team that cannot explain code flagged as similar to an external source, in the defense, will be treated as an integrity case, not merely graded down.